Chapter 8

Support Vector Machines

Assoc. Prof. Dr. Duong Tuan Anh
Faculty of Computer Science and Engineering,
HCMC Univ. of Technology
3/2015

Outline

- 1. Introduction
- 2. Support vector machine for binary classification
- 3. Multi-class classifiction
- 4. Learning with soft margin
- 5. SVM Tools
- 6. Applications of SVMs

1.Introduction

- A new method for the classification of both linear and nonlinear data.
- SVM is an algorithm that works as follows:
  - It uses a nonlinear mapping to transform the original training data into a higher dimension.
  - Within this new dimension, it searches for the linear *optimal separating hyperplane*.
  - With an appropriate nonlinear mapping to a sufficiently high dimension, data from two classes can be separated by a hyperplane.
  - SVM finds this hyperplane using *support vectors* (“essential” training examples) and *margins* (defined by the support vectors)

Introduction (cont.)

- The first paper on SVM: 1992 by V. Vapnik et al.
- Although the training time of SVMs can be very slow, they are highly accurate.
- SVMs are much less prone to *overfitting*.
- SVMs can be used for prediction and classification.
- SVMs can be applied to:
  - Handwritten digit recognition, object recognition, speaker identification, time series prediction.

2. Support vector machine for binary classification

The case when the data are *linearly separable*

- Example: Two-class problem, linearly separateble.
- $D = \{(\boldsymbol{X}_1, y_1), (\boldsymbol{X}_2, y_2), \ldots, (\boldsymbol{X}_m, y_m)\}$
  - $\boldsymbol{X}_i$ is a training tuple, y_i is associated class label.
  - $y_i \in \{+1, -1\}$
  - Assume each tuple has two input attributes A_1, A_2.
- From the graph, the 2-D data are *linearly separatable.*
  - There are an infinite number of separating lines that could be drawn to separate all of the tuples of class +1 from all of the tuples of class -1.

2.1 The case of Linearly separable data

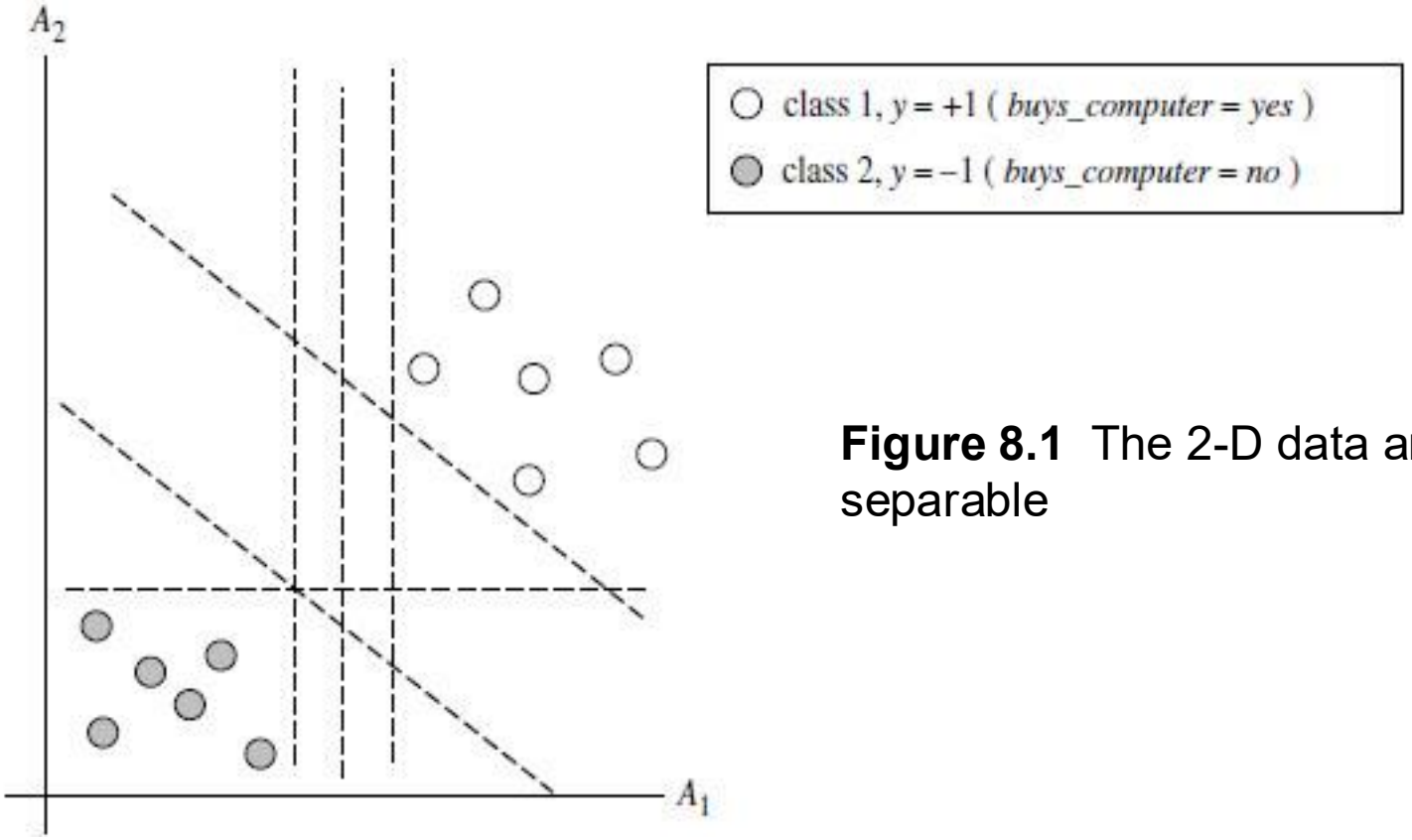
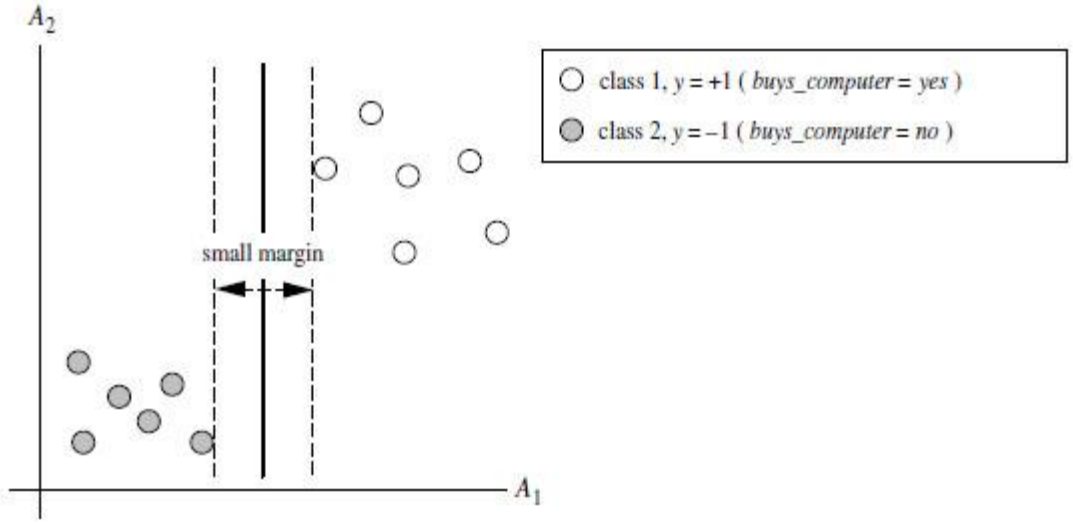


Figure 8.1 The 2-D data are linearly separable

There are an infinite number of *separating hyperplanes* or "*decision boundaries*". Which one is best?

SVM - Maximum marginal hyperplane

- Generalizing to *n* dimensions, we want to find the best hyperplane.
- *Hyperplane* = "*decision boundary*"
- How can we find the best hyperplane?
- An SVM approaches this problem by searching for the *maximum marginal hyperplane*.
- The following figure shows two possible separating hyperplanes and their associated *margins*.
  - Both hyperplanes can correctly classify all of the given data tuples.
  - However, we expect the hyperplane with the larger margin to be more accurate at classifying future data tuples than the hyperplane with smaller margin.



Two possible separating hyperplanes and their associated margins. Which one is better? The one with the *larger margins* should have *greater* generalization accuracy.

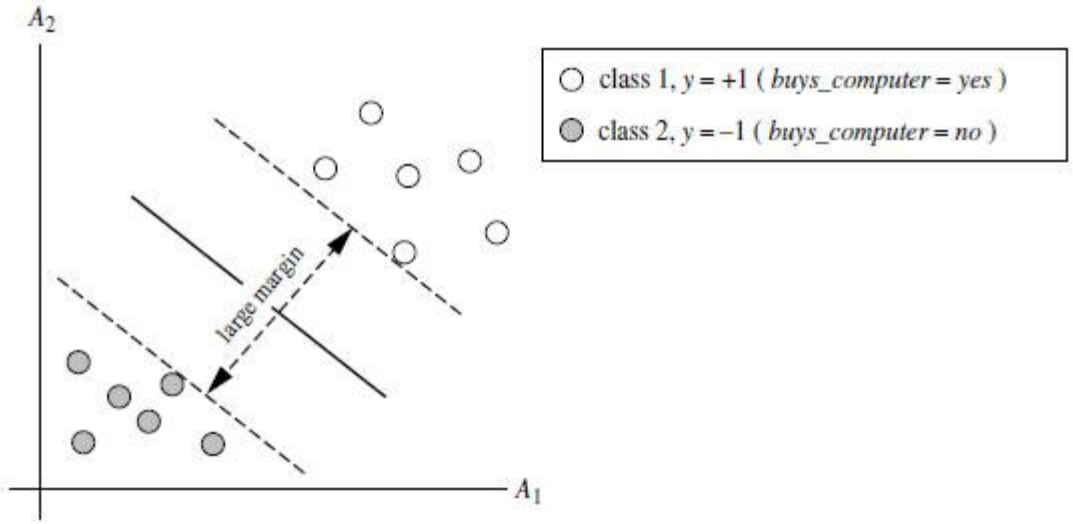


Figure 8.2 Two possible hyperplanes and their associated margins

Margin

- The SVM searches for the hyperplane with the largest margin (*maximum marginal hyperplane - MMH*).
- What is *margin*?
- The *shortest distance* from a hyperplane to one *side* of its margin is equal to the shortest distance from the hyperplane to the other *side* of its margin.
- When dealing with the MMH, this distance is the shortest distance from the MMH to the closest training tuple of either class.

Maximum margin

- A ***separating hyperplane*** can be writen as

$$\boldsymbol{W}.\boldsymbol{X} + b = 0 \qquad (1)$$

where $\boldsymbol{W}$ is a weight vector, $\boldsymbol{W} = \{w_1, w_2, \ldots, w_n\}$; n is the number of attributes; and b is a scalar (a *bias*).

- Let's consider two input attributes, A_1 and A_2. Training tuples are 2-D. $\boldsymbol{X} = (x_1, x_2)$.
- We can rewrite the above separating hyperplane as:

$$w_0 + w_1 x_1 + w_2 x_2 = 0 \qquad (2)$$

- Any point that lies above the separating hyperplane satisfies

$$w_0 + w_1 x_1 + w_2 x_2 > 0$$

- Any point that lies below the separating hyperplane satisfies

$$w_0 + w_1 x_1 + w_2 x_2 < 0$$

Maximum margin (cont.)

- The weights can be adjusted so that the hyperplanes defining the "sides" of the margin (called ***support hyperplanes***) can be rewritten as

  H_1: $w_0 + w_1x_1 + w_2x_2 = 1$ for $y_i = +1$

  H_2: $w_0 + w_1x_1 + w_2x_2 = -1$ for $y_i = -1$

  That is, any tuple that falls on or above H_1 belongs to class +1. These tuples satisfy the constraint:

  $$w_0 + w_1x_1 + w_2x_2 \geq 1 \text{ for } y_i = +1 \qquad (3)$$

  And any tuple that falls on or below H_2 belongs to class -1. These tuples satisfy the constraint:

  $$w_0 + w_1x_1 + w_2x_2 \leq -1 \text{ for } y_i = -1 \qquad (4)$$

- Combining (3) and (4), we get:

  $$y_i(w_0 + w_1x_1 + w_2x_2) \geq 1, \forall i. \qquad (5)$$

  All the tuples in the training set satisfy the constraint (5).

- Any training tuples that fall on the support hyperplanes (H_1 and H_2) are called ***support vectors***. These tuples satisfy the equation: $y_i(w_0 + w_1x_1 + w_2x_2) = 1, \forall i$.

Support vectors

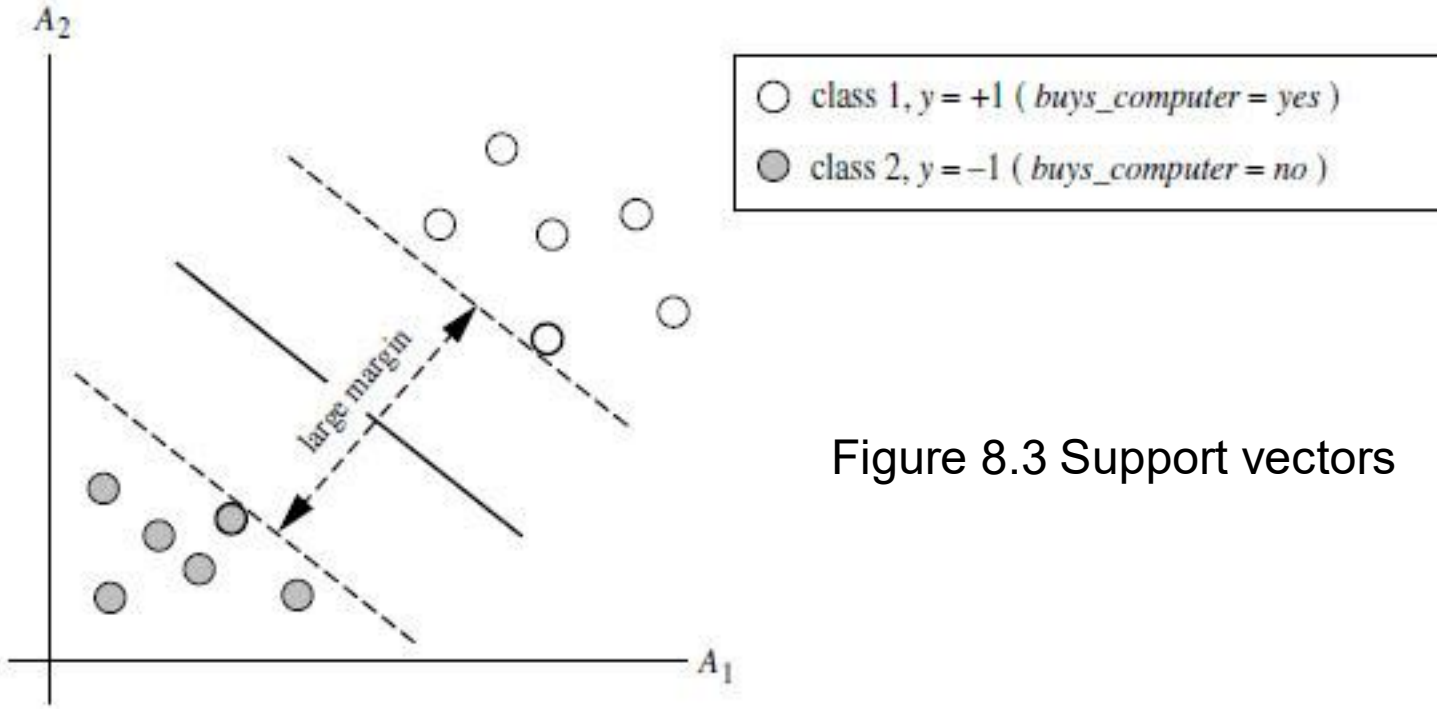


Figure 8.3 Support vectors

The ***support vectors*** are the most difficult tuples to classify and give the most information regarding classification.

Support vectors (cont.)

- The distance from the separating hyperplane to any point on H_1 is $1/||\boldsymbol{W}||$

  where $||\boldsymbol{W}||$ is the Euclidean norm of $\boldsymbol{W}$, that is

  $$sqrt(w_1^2 + w_2^2 + \ldots + w_n^2).$$

Why the distance from support vector to the separating hyperplane = 1/||*W*|| ?

- Let us focus on the two-class case and linearly separable data. The decision surface has the equation:

  $\boldsymbol{W}.\boldsymbol{X} + w_0 = 0$ (C1)

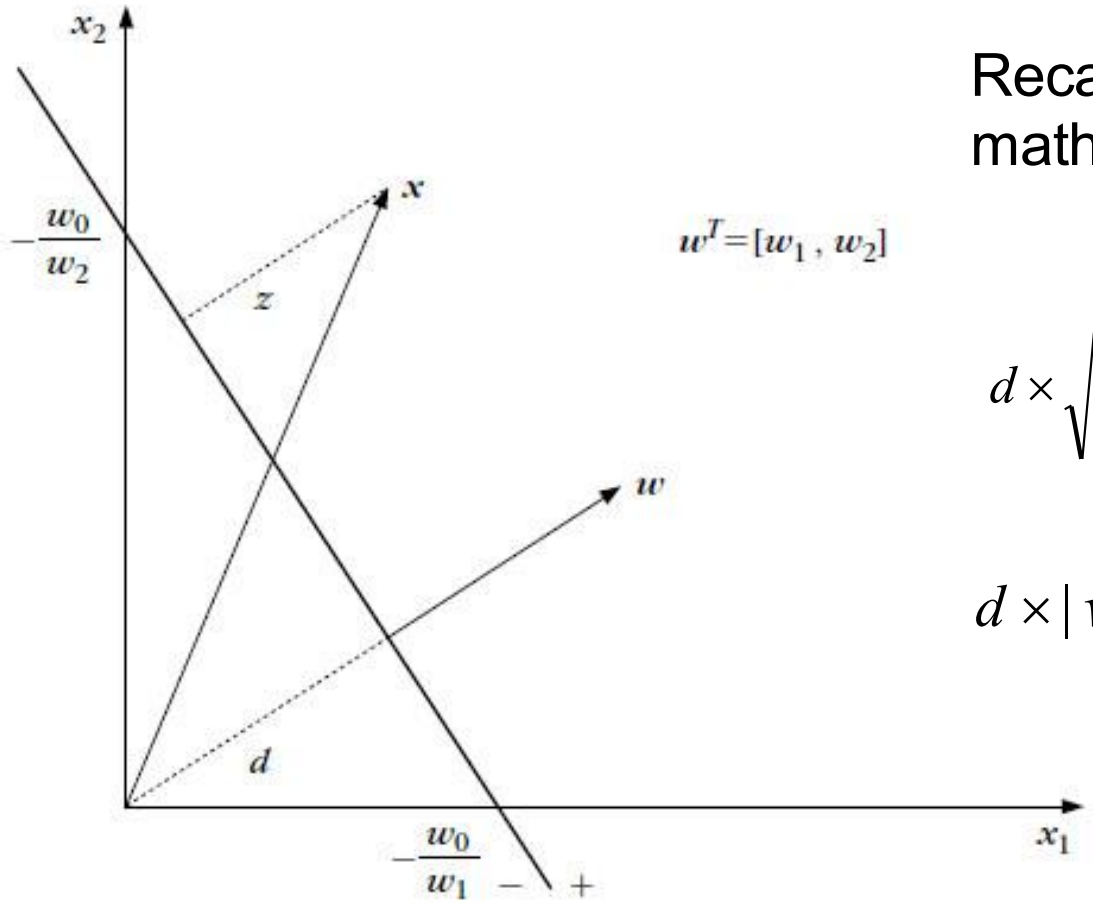
  where $\boldsymbol{W}$ is a weight vector, $\boldsymbol{W} = \{w_1, w_2, \ldots, w_n\}$; n is the number of attributes; and w_0 is a scalar (a *bias*).
- If $\boldsymbol{X_1}$ and $\boldsymbol{X_2}$ are two points on the decision hyperplane, the following is valid

  $0 = \boldsymbol{W}.\boldsymbol{X_1} + w_0 = \boldsymbol{W}.\boldsymbol{X_2} + w_0$

  $\Rightarrow \boldsymbol{W}(\boldsymbol{X_1} - \boldsymbol{X_2}) = 0$ (C2)

  Since the difference vector $\boldsymbol{X_1} - \boldsymbol{X_2}$ lies on the decision hyperplane (for any $\boldsymbol{X_1}, \boldsymbol{X_2}$), it is clear from (C2) that the vector $\boldsymbol{W}$ is *orthogonal to* the decision hyperplane.

The figure shows the case of 2-D data with $w_1>0$, $w_2 > 0$, $w_0 <0$. Let d be the distance from the origin to the decision hyperplane and z be the distance from a point $\boldsymbol{x}$ to the decision hyperplane



Recalling our high school math, it is easy to obtain:

$$d \times \sqrt{(\frac{-w_0}{w_1})^2 + (\frac{-w_0}{w_2})^2} = (-\frac{w_0}{w_1})(-\frac{w_0}{w_2})$$

$$d \times |w_0| \times \sqrt{\frac{1}{(w_1)^2} + \frac{1}{(w_2)^2}} = \frac{(w_0)^2}{w_1 w_2}$$

We derive:

$$d \times \sqrt{(w_1)^2 + (w_2)^2} = |\, w_0 \,|$$

$$d = \frac{|\, w_0 \,|}{\sqrt{(w_1)^2 + (w_2)^2}}$$

From the figure, it is clear that *z* is proportional to *d*. So we can write:

$$z = \frac{|\, g(x) \,|}{\sqrt{(w_1)^2 + (w_2)^2}}$$

$$z = \frac{|\, g(x) \,|}{\| w \|}$$

$|g(\boldsymbol{x})|$ is a quantity that is dependent on the Euclidean distance of the point $\boldsymbol{x}$ from the decision hyperplane. On one side of the plane, $g(\boldsymbol{x})$ takes positive values and on the other negative.

- Recall that the distance of a point ***x*** from a decision hyperplane is given by:

$$z = \frac{|g(x)|}{||w||}$$

We can now scale ***w***, w_0 so that the value of $g(\boldsymbol{x})$, at the nearest points in class +1 and class -1, is equal to 1 for data points in class +1 and, thus, equal to -1 for data points in class -1.

This is equivalent to

Having a margin of $1/||\boldsymbol{w}||$

Related optimization problem

- By definition, the distance from the separating hyperplane to any point on H_1 is equal to the distance from any points in H_2 to the hyperplane.
- Therefore, the maximum margin is $2/||\boldsymbol{W}||$
- How does an SVM find the MMH and the support vectors?
  - Maximizing the margin is equivalent to minimizing

$$(1/2)||\boldsymbol{W}||^2 \qquad (6)$$

    subject to the constraints:

$$y_i(\boldsymbol{w}.\boldsymbol{x_i} + b) \geq 1 \quad \forall i \qquad (7)$$

  - This is a *constrained optimization* problem in which we minimize an objective function (6) subject to the constraints (7).

Related optimization problem

- Lagrange formulation:

$$L(\mathbf{w}, b) = \frac{1}{2}(\mathbf{w} \cdot \mathbf{w}) - \sum_{i=1}^{m} \alpha_i \, (y_i (\mathbf{w} \cdot \mathbf{x}_i + b) - 1) \tag{8}$$

where α_i are Lagrange multipliers, $\alpha_i \geq 0$. To solve the optimization problem, we compute the derivatives w.r.t. *b* and ***w*** and set these to zero:

$$\frac{\partial L}{\partial b} = -\sum_{i=1}^{m} \alpha_i y_i = 0 \tag{9}$$

$$\frac{\partial L}{\partial \mathbf{w}} = \mathbf{w} - \sum_{i=1}^{m} \alpha_i y_i \mathbf{x}_i = 0 \tag{10}$$

Substituting ***w*** from (10) back to *L*(***w***, *b*), we get the dual formulation:

$$W(\alpha) = \frac{1}{2}\left(\sum_{i=1}^{m}\alpha_i y_i x_i\right)\left(\sum_{i=1}^{m}\alpha_i y_i x_i\right)$$

$$-\left(\sum_{i=1}^{m}\alpha_i y_i x_i\right)\left(\sum_{i=1}^{m}\alpha_i y_i x_i\right) - b\sum_{i=1}^{m}\alpha_i y_i + \sum_{i=1}^{m}\alpha_i$$

$$W(\alpha) = \sum_{i=1}^{m}\alpha_i - \frac{1}{2}\left(\sum_{i=1}^{m}\alpha_i y_i x_i\right)\left(\sum_{i=1}^{m}\alpha_i y_i x_i\right)$$

W(α) can be reformulated as:

$$W(\alpha) = \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{m} \alpha_i \alpha_j y_i y_j \left(\mathbf{x}_i \cdot \mathbf{x}_j\right) \qquad (11)$$

which must be maximized w.r.t the α_i subject to the constraints:

$$\alpha_i \geq 0 \qquad \sum_{i=1}^{m} \alpha_i y_i = 0 \qquad (12)$$

The dual formulation has the objective function that is quadratic in the parameters α_i. This is a ***Quadratic Programming*** problem.

Related optimization problem

- *A constrained (convex) quadratic optimization problem*.

  $\Rightarrow$ Lagrange formulation.
- Solving the optimization problem by **Karush-Kuln-Tucker** (KKT) conditions (traditional method).
- If the data is small (<2000 training tuples), any optimization software package for solving constrained convex quadratic optimization problem can be used.
- For larger data, special and more efficient training algorithm for training SVMs can be used (e.g. **Sequential Minimal Optimization**–SMO algorithm).
- Once we've found the support vectors and MMH, we have a trained support vector machine.
- The MMH is *linear class boundary*, and so the corresponding SVM can be used to classify linear separable data.

How to use trained SVM to classify test data

Based on Lagrangian formulation, the MMH can be rewritten as the decision boundary with the equation:

$$d(X^T) = \sum_{i=1}^{l} y_i \alpha_i X_i X^T + b_0, \qquad (13)$$

where y_i is the class label of support vector X_i, X^T is a test tuple, α_i and b_0 are numeric parameters that were determined automatically by the optimization or SVM algorithm above, and l is the number of support vectors.

Note: Eq (13) derives from the substitution of Eq. (10) to Eq. (1).

For linearly separable data, the support vectors are a subset of the training tuples.

Given a test tuple X^T, we plug it into Eq. (13), and then check to see the sign of the result. This tell us on which side of the hyperplane the test tuple falls.

If the sign is positive, the X^T falls on or above the MMH, and so SVM predicts that X^T belongs to the class +1. If the sign is negative, then X^T falls on or below the MMH and the class prediction is -1.

Some good characteristics of SVM

- The Lagrangian formulation (Eq. (13)) contains a dot product between support vector $\boldsymbol{X}_i$ and test tuple $\boldsymbol{X}^T$.
- The complexity of the learned classifier is characterized by the number of support vectors rather than the dimensionality of the data.
- SVMs tend to be less prone to *overfitting* than some other methods.
- The support vectors are the *essential training tuples*.
- The number of support vectors can be used to compute an (upper) bound on the expected error rate of the SVM classifier, which is independent of the data dimensionality.

Example

Consider the three two-dimensional data points where (1,1) and (2,2) are from class *X* and (2,0) is from class *O*. Here $\boldsymbol{w}$ is a two-dimensional vector and so the objective function and the three constraints are:

Minimize $(1/2)\ ||\boldsymbol{w}||^2$

such that $\boldsymbol{w}.\boldsymbol{x} + b \geq 1 \quad \forall\ \mathbf{x} \in O$ and

$\boldsymbol{w}.\boldsymbol{x} + b \leq -1\ \forall\ \mathbf{x} \in X$.

Note that the 2nd constraint is $\boldsymbol{w}.\boldsymbol{x} + b \leq -1$ where $\boldsymbol{x} = (1,1)$. So we get $w_1 + w_2 + b \leq -1$.

By writing the constraints in terms of the two components of $\boldsymbol{w}$, namely w_1 and w_2 and the corresponding components of $\boldsymbol{x}$, we have the Lagrangian form (see Eq. 8) to be

$$J(\boldsymbol{w}) = (1/2)||\boldsymbol{w}||^2 - \alpha_1(-w_1 - w_2 - b - 1) - \alpha_2(-2w_1 - 2w_2 - b - 1) - \alpha_3(2w_1 + b - 1)$$

- Differentiating $J(\boldsymbol{w})$ with respect to w_1, w_2 and equating to 0, we get

  $w_1 = -\alpha_1 - 2\alpha_2 + 2\alpha_3$

  $w_2 = -\alpha_1 - 2\alpha_2$

- Differentiating $J(\boldsymbol{w})$ with respect to b and equating to 0, we get

  $\alpha_1 + \alpha_2 - \alpha_3 = 0$

- Differentiating $J(\boldsymbol{w})$ with respect to α_i and equating to 0, we get

  $-w_1 - w_2 - b - 1 = 0$

  $-2w_1 - 2w_2 - b - 1 = 0$

  $2w_1 + b - 1 = 0$

From the three last equations, we get $w_2 = -1$, $w_1 = 1$ and $b = -1$. That means

$$w = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

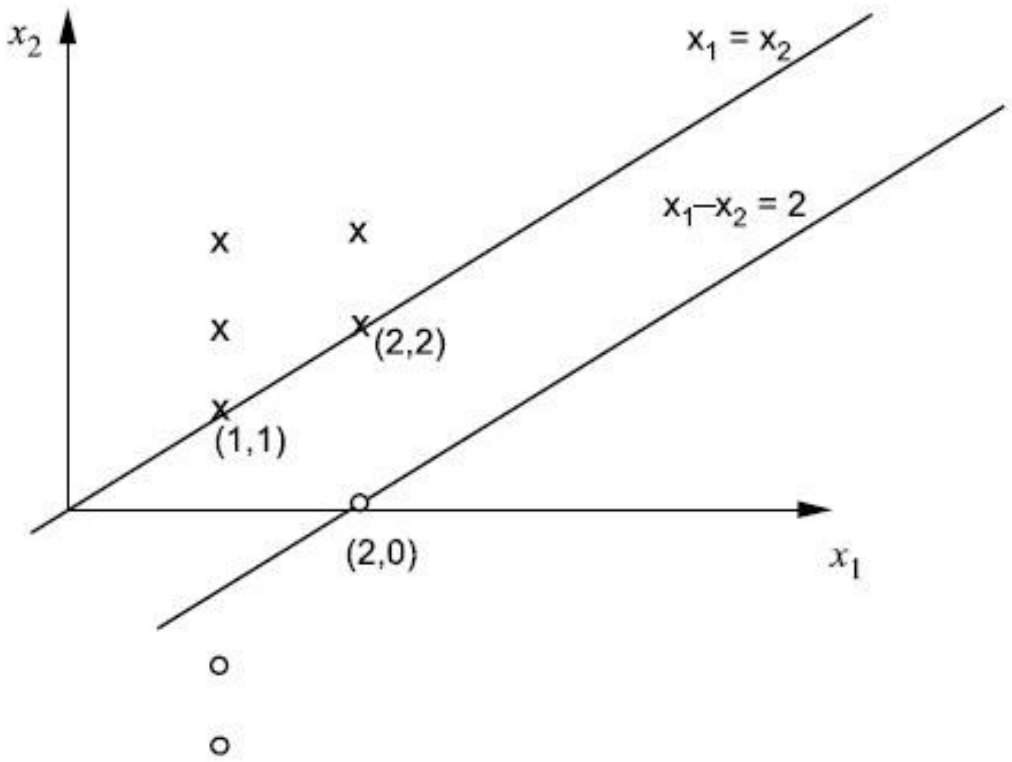
Note that in this case, all the given data vectors are *support vectors*.

- Using the three first equations, it is possible to solve to find α_1, α_2 *and* α_3 . The solution is

  $\alpha_1 = 1$, $\alpha_2 = 0$ *and* $\alpha_3 = 1$.
- So, the equation for the MMH is: ***w.x*** + *b* = 0

  i.e. the line $x_1 - x_2 - 1 = 0$
- The α_1 α_2 α_3 will be needed when classifying a test pattern

Figure 8.4



2.2 The case of linearly inseparable data

- The linear SVMs we studied would not be able to find a feasible solution when the data is linearly inseparable.
- The approach: Linear SVMs can be **extended** to create nonlinear SVMs for classification of linearly inseparable data.
- Such SVMs are capable of finding *nonlinear decision boundaries* (i.e. nonlinear hyperplanes) in input space.

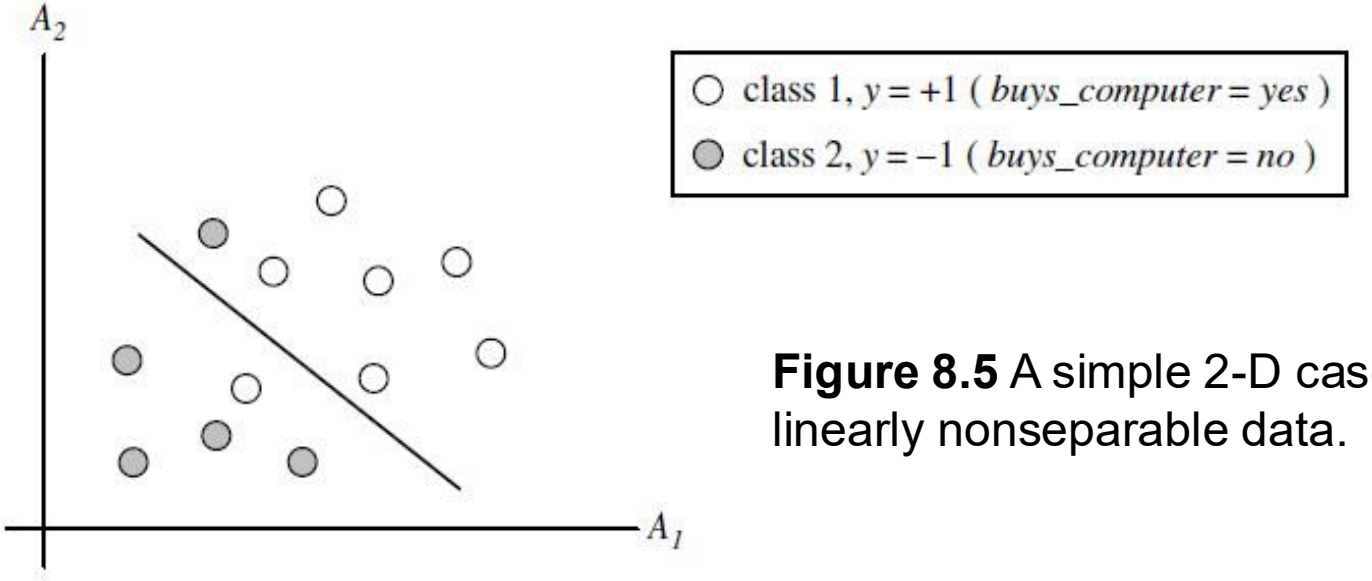


Figure 8.5 A simple 2-D case showing linearly nonseparable data.

- How can we extend the linear approach?
  - In the first step, we *transform* the original input data into a higher dimensional space using a nonlinear mapping.
  - Once the data have been transformed into the new higher space, the second step searches for a *linear separating hyperplane* in the new space.
  - We again end up with a quadratic optimization problem that can be solved using the linear SVM formulation. The *maximum marginal hyperplane* found in the new space corresponds to a *nonlinear separating hypersurface* in the original space.

Nonlinear mapping

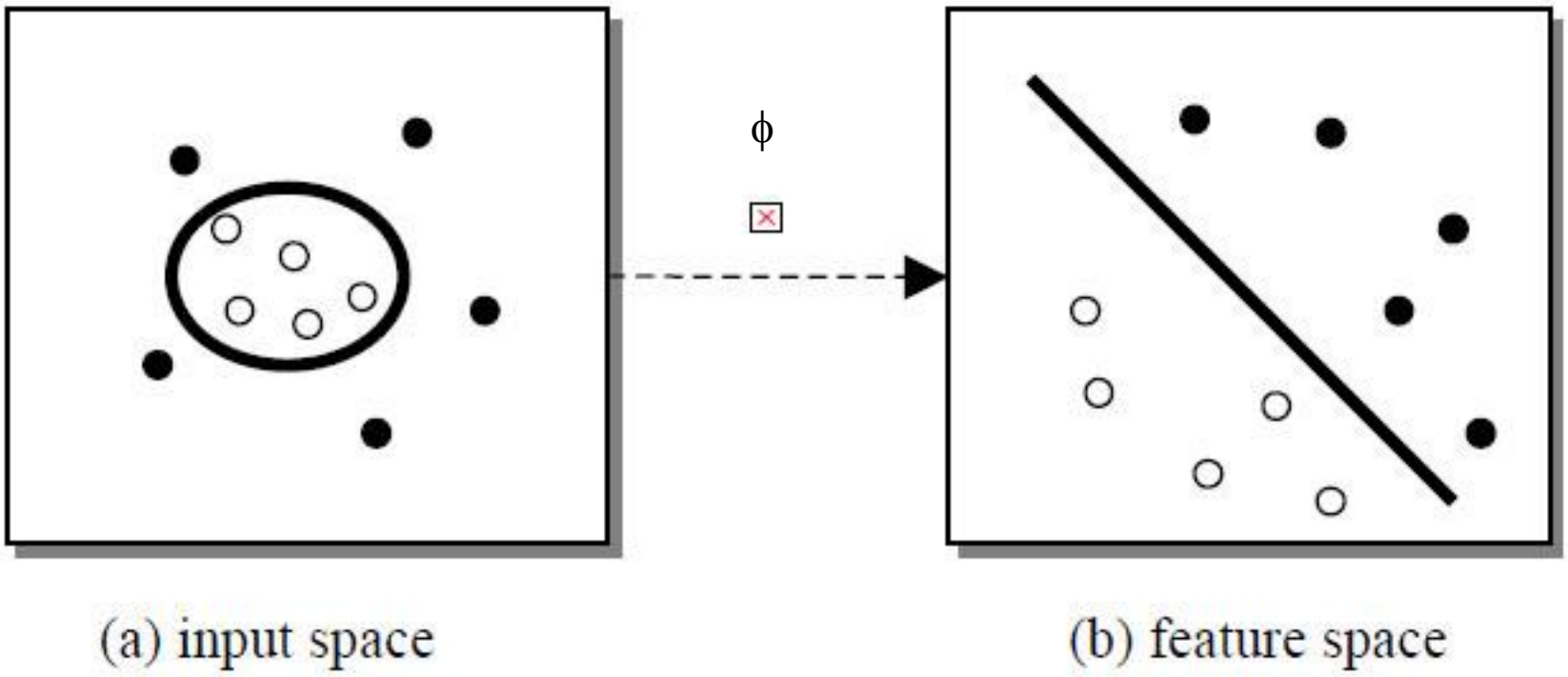


Figure 8.6 Feature space is related to the input space via a nonlinear mapping

Example: Nonlinear mapping of original input data into a higher dimensional space

- A 3-D input vector $\boldsymbol{X} = (x_1, x_2, x_3)$ is mapped into a 6-D space, $\boldsymbol{Z}$, using the mappings $\phi_1(\boldsymbol{X}) = x_1$, $\phi_2(\boldsymbol{X}) = x_2$, $\phi_3(\boldsymbol{X}) = x_3$, $\phi_4(\boldsymbol{X}) = (x_1)^2$, $\phi_5(\boldsymbol{X}) = x_1x_2$, $\phi_6(\boldsymbol{X}) = x_1x_3$
- A decision hyperplane in the new space is $d(\boldsymbol{Z}) = \boldsymbol{WZ} + b$, where $\boldsymbol{W}$ and $\boldsymbol{Z}$ are vectors. This is linear.
- We solve for $\boldsymbol{W}$ and b and then substitute back so that the linear decision hyperplane in the new ($\boldsymbol{Z}$) space corresponds to a nonlinear second-order polynomial in the original 3-D input space.

$$d(\boldsymbol{Z}) = w_1x_1 + w_2x_2 + w_3x_3 + w_4(x_1)^2 + w_5x_1x_2 + w_6\,x_1x_3 + b$$
$$= w_1z_1 + w_2z_2 + w_3z_3 + w_4z_4 + w_5z_2 + w_6z_6 + b$$

- But there are some problems.

Nonlinear mapping

- 1. How do we choose the nonlinear mapping to a higher dimensional space?
- 2. The computation involved will be costly.
  - In Eq. (13), for a test tuple $\boldsymbol{X}^T$, we have to compute its dot product with every one of the support vectors.
  - In training, we have to compute a similar dot product several times in order to find the MMH.
  - This is expensive.
- To overcome these problems, we can use a math trick.

Math trick

- In solving the quadratic optimization problem of the linear SVM (i.e. when searching for a linear SVM in the new higher dimensional space), the training tuples appear only in the form of dot product: $\phi(\boldsymbol{X}_i).\ \phi(\boldsymbol{X}_j)$, where $\phi(\boldsymbol{X})$ is the nonlinear mapping.
- Instead of computing the dot product on the transformed data tuples it is mathematically equivalent to apply a ***kernel function***, $K(\boldsymbol{X}_i, \boldsymbol{X}_j)$ to the original input data.

$$K(\boldsymbol{X}_i, \boldsymbol{X}_j) = \phi(\boldsymbol{X}_i).\ \phi(\boldsymbol{X}_j) \qquad (14)$$

- Everywhere that $\phi(\boldsymbol{X}_i).\ \phi(\boldsymbol{X}_j)$ appears in the training algorithm, we can replace it with $K(\boldsymbol{X}_i, \boldsymbol{X}_j)$. And all calculations are made in the original input space.
- We can avoid the nonlinear mapping. We pay attention to kernel functions.

Kernel functions

- After applying the trick, we can proceed to find the maximal seperating hyperplane. The procedure is similar to that for linear SVMs.
- It involves a user-specified upper bound *C* on the Lagrange multipliers α_i. This upper bound is determined experimentally.

- What are the kernel functions that could be used?
- Three admissible kernel functions include:

1. **Polynomial kernel of degree *h***: $K(\boldsymbol{X}_i, \boldsymbol{X}_j) = (\boldsymbol{X}_i.\boldsymbol{X}_j + 1)^h$

   where *h* is selected by the user.

Example: $h = 2$ and the dimension is 2,

$$K(\boldsymbol{X}, \boldsymbol{Y}) = (x_1y_1 + x_2y_2 + 1)^2 = 1 + 2x_1y_1 + 2x_2y_2 + 2x_1x_2y_1y_2 + x_1^2y_1^2 + x_2^2y_2^2$$

Kernel functions (cont.)

2. **Radial-basis function**

$$K(X_i, X_j) = e^{-\|X_i - X_j\|^2 / 2\sigma^2}$$

which defines a spherical kernel as where $\boldsymbol{X_i}$ is the ***center*** and σ, supplied by the user, defines the ***radius***. This is similar to radial basis function that we discuss in the Chapter 7 (Neural Networks).

3. **Sigmoidal function**

$$K(\boldsymbol{X}_i, \boldsymbol{X}_j) = tanh(\kappa \boldsymbol{X}_i . \boldsymbol{X}_j - \sigma)$$

where *tanh*(.) has the same shape with *sigmoid*, except that it ranges between -1 and 1. This is similar to multilayer neural networks that we discuss in chapter 7.

For binary classification with a given choice of kernel, the learning task involves maximization of:

$$W(\alpha) = \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{m} \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j) \qquad (15)$$

subject to the constraints (12). The bias *b* can be found as follows.

For a training tuple with y_i = +1, we note that:

$$\min_{\{i|y_i=+1\}} [\mathbf{w} \cdot \mathbf{x}_i + b] = \min_{\{i|y_i=+1\}} \left[\sum_{j=1}^{m} \alpha_j y_j K(\mathbf{x}_i, \mathbf{x}_j) \right] + b = 1 \qquad (16)$$

Using (10) and with a similar expression for the trainining tuples labeled y_i = -1. From this observation, we deduce:

$$b = -\frac{1}{2} \left[\max_{\{i|y_i=-1\}} \left(\sum_{j=1}^{m} \alpha_j y_j K(\mathbf{x}_i, \mathbf{x}_j) \right) + \min_{\{i|y_i=+1\}} \left(\sum_{j=1}^{m} \alpha_j y_j K(\mathbf{x}_i, \mathbf{x}_j) \right) \right] \qquad (17)$$

How to train and use an SVM binary classifier

To construct an SVM binary classifier, we place the training data (x_i, y_i) into Eq (15) and maximize $\boldsymbol{W}(\alpha)$ subject to (12). From the optimal values of α_i, which we denote α^*, we calculate the bias b from (17).

For a new input vector $\boldsymbol{z}$, the predicted class is based on the sign of:

$$\sum_{i=1}^{m} \alpha_i^* y_i K(x_i, z) + b^* \qquad (18)$$

where b^* denotes the value of the bias at optimality.

This expression derives from the substitution of $\boldsymbol{w}^*$ from (10) into the decision function (1), i.e. $f(\boldsymbol{z}) = \text{sign}(\boldsymbol{w}^*.\phi(z) + b^*)$

$$= \text{sign}(\Sigma \alpha_i^* y_i \phi(x_i).\phi(z) + b^*)$$

$$= sign(\sum_i \alpha_i^* y_i K(x_i, z) + b^*)$$

Sometimes, we refer to (α^*, b^*) as a *hypothesis* modelling the data.

Support vectors and non-support vectors

- When the MMH is found in feature space, only those tuples which lie closest to the hyperplane have α_i^* >0 and these tuples are the *support vectors*.
- All the other points have α_i^* = 0, and the decision function is independent with these points.
- If we remove some of the non-support vectors, the current separating hyperplane would remain uneffected.

Kernel functions and SVM

- Each of the three kernel functions results in a different *nonlinear classifier* in the (original) input space.
- There are no rules for determining which kernel will result in the most accurate SVM.
- In practice, the kernel chosen does not make a large difference in resulting accuracy.
- SVM training always finds a *global solution*; unlike neural networks, where many local minima usually exist.
- A major research goal in SVMs is to improve the speed in training and testing so that SVMs may becomes a more feasible option for large datasets.
- Other issues include determining the *best kernel* for a given dataset.

3. SVM as Multiclass classifier

- Binary nonlinear SVMs for binary classification have been described.
- SVM classifiers can be combined for the multi-class case.
- Given *m* classes, trains *m* classifiers, one for each class (where classifier *j* learns to return a positive value for class *j* and a negative value for the rest).
- A test tuple is assigned the class corresponding to the largest positive distance.

$$d_j(\boldsymbol{x}) = \boldsymbol{w_j}.\boldsymbol{x} + b_j \qquad (\text{SVM}_j)$$

4. Learning with noise: soft margins

- In practical applications, datasets contain noise and the two classes are not completely separable, but a hyperplane that maximize the margin while minimizing a *quantity* proportional to the mis-classification error can still be determined.

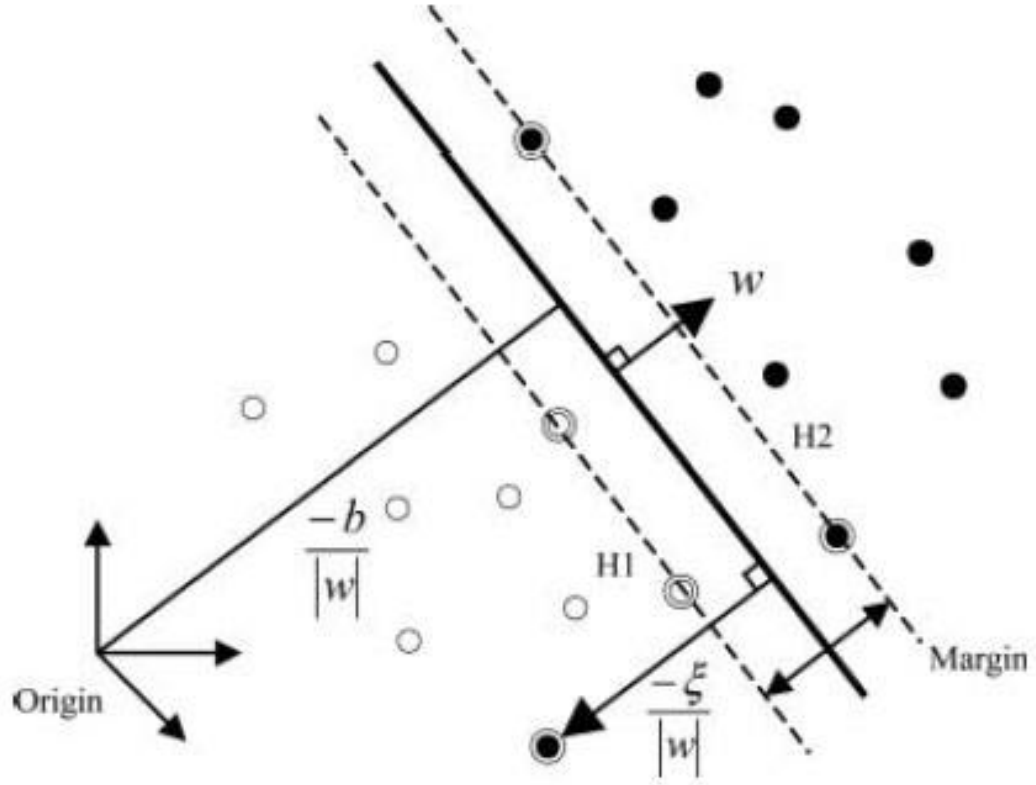


Figure 8.7

- This can be done by introducing non-negative *slack variables* ξ_i in constraint (7), which becomes:

$$y_i(\boldsymbol{w}.\ \phi(\boldsymbol{x}_i) + b) \geq 1 - \xi_i\ ,\ \forall i \qquad (19)$$

- If an error occurs, the corresponding ξ_i must exceed 1, so $\sum_1^m \xi_i$ is an upper bound for the number of misclassification errors. Here the objective function to be minimized can changed into:

$$\min\left[\frac{1}{2} w.w + C\sum_{i=1}^{m} \xi_i\right] \qquad (20)$$

where ***C* is a parameter chosen by the user that controls the *tradeoff* between the margin and the misclassification errors**. A larger *C* means that a higher ***penalty*** to misclassification errors is assigned.

Minimizing (20) under constraint (19) gives the *generalized separating hyperplanes*. This still remains a *quadratic programming* problem. We can pose the dual problem (using Lagrange multipliers) and solve it.

SVM with soft margins

- Finally, the dual problem to be solved is

$$\max_{\alpha_i} \left\{ \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i=1}^{m} \sum_{j=1}^{m} \alpha_i (y_i y_j K(x_i, x_j)) \alpha_j \right\}$$

subject to the constraints

$$0 \le \alpha_i \le C, \ \sum_{i=1}^{m} \alpha_i y_i = 0$$

and the classifier becomes

$$f(x) = sign\left(\sum_{i=1}^{m} (\alpha_i y_i K(x, x_i) + b \right)$$

How to solve the Quadratic Programming Problem

- Traditional QP algorithms are not suitable for large size problems.
- A number of algorithms have been suggested to solve the dual problems.
- **Osuma** et al. proved a theorem which proves that the large QP problem can be broken down into a series of smaller QP sub-problems.
- **Pratt** proposed a *Sequential Minimal Optimization* (*SMO*) to quickly solve SVM QP problem without any extra matrix storage and without using numerical QP optimization steps at all. Using Osuma's theorem to ensure convergence, SMO decomposes the overall QP problem into QP sub-problems. SMO chooses to solve the smallest possible optimization problems at every step. At each step:
  - (1) SMO chooses two Lagrange multipliers to jointly optimize.
  - (2) Finds the optimal values for these multipliers, and
  - (3) Updates the SVMs to reflect the new optimal values.

6. SVM tools

- In WEKA, the data mining software, there are ANN and SVM tools for classification.
- LIBSVM – A library for support vector machines (C. C. Chang & C. J. Lin)

  http://www.csie.ntu.edu.tw/~cjlin/libsvm/
- Support vector machine – MATLAB
- Support vector machine tool, **SVM^{light}** (by T. Joachims).
- SVM tools in R.
- Scikit-learn

7. Applications of SVMs

- Face detection
- Face recognition
- Object recognition
- Handwritten character/digit recognition
- Speech recognition
- Image retrieval
- Prediction
  - Financial time series prediction
  - Bankruptcy prediction
- Anomaly detection in time series data

Reference

- J. Han & M. Kamber, Data Mining: Concepts and Techniques, 2nd Edition, Morgan Kaufmann Publisher, CA, 2006.
- C. Campbell & Y. Ying, Learning with Support Vector Machines, Morgan & Claypool Publishers, 2011.
- F.E.H. Tay & L. Cao, Application of Support Vector Machines in Financial Time Series Forecasting, *Omega, The International Journal of Management Science*, Vol. 29, 2001, pp. 309-317.

Appendix: Method of Lagrange Multipliers for constrained optimization

- A method for obtaining the relative maximum or minimum values of a function $F(x,y,z)$ subject to a constraint condition $\phi(x,y,z)=0$ consists of the formation of the auxiliary function

$$G(x,y,z) \equiv F(x,y,z) + \lambda\phi(x,y,z) \qquad \text{(A1)}$$

subject to the conditions:

$$\frac{\partial G}{\partial x} = 0, \frac{\partial G}{\partial y} = 0, \frac{\partial G}{\partial z} = 0 \qquad \text{(A2)}$$

which are necessary conditions for a relative maximum or minimum. The parameter λ which is independent of x, y, z is called a *Lagrange multiplier*.

The condition (A2) are equivalent to $\nabla G = 0$

And hence: $\nabla F + \lambda \nabla \phi = 0$

(refer to Introduction to Data Mining)

- The method can be generalized. If we wish to find the relative maximum or minimum values of a function $F(x_1,x_2,\ldots,x_n)$ subject to the constraint conditions $\phi_1(x_1,x_2,\ldots,x_n) = 0$, $\phi_2(x_1,x_2,\ldots,x_n) = 0,\ldots,\ \phi_k(x_1,x_2,\ldots,x_n) = 0$, we form the auxiliary function:

$$G(x_1,x_2,\ldots,x_n) \equiv F + \lambda_1\phi_1 + \lambda_2\phi_2 + \ldots + \lambda_k\phi_k$$

subject to the necessary conditions

$$\frac{\partial G}{\partial x_1} = 0, \frac{\partial G}{\partial x_2} = 0, \ldots \frac{\partial G}{\partial x_n} = 0$$

where $\lambda_1, \lambda_2, \ldots, \lambda_k$ which are independent of $x_1, x_2, \ldots, x_n$ are called *Lagrange multipliers*.

Quadratic Programming

- The quadratic programming differs from the linear programming only in that the objective function also include x_j^2 and $x_i x_j$ ($i \neq j$) terms.
- If we used matrix notation, the problem is to find $\boldsymbol{x}$ so as to

  Maximize $f(\boldsymbol{x}) = \boldsymbol{cx} - (1/2)\boldsymbol{x}^T\boldsymbol{Q}\boldsymbol{x}$,

  subject to $\boldsymbol{Ax} \leq \boldsymbol{b}$ and $\boldsymbol{x} \geq 0$

  where $\boldsymbol{c}$ is a row vector, $\boldsymbol{x}$ and $\boldsymbol{b}$ are column vectors, $\boldsymbol{Q}$ and $\boldsymbol{A}$ are matrices. The q_{ij} (elements of $\boldsymbol{Q}$) are given constants such that $q_{ij} = q_{ji}$.
- By performing vector and matrix multiplications, the objective function is expressed in terms of these q_{ij}, the c_j (elements of $\boldsymbol{c}$) and the variables as follows:

$$f(\mathbf{x}) = \boldsymbol{cx} - (1/2)\boldsymbol{x}^T\boldsymbol{Q}\boldsymbol{x}$$

$$= \sum_{j=1}^{n} c_j x_j - \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} q_{ij} x_i x_j$$

Example of Quadratic Programming

- Consider an example of a QP problem

  Maximize $f(x_1,x_2) = 15x_1 + 30x_2 + 4x_1x_2 - 2x_1^2 - 4x_2^2$,

  subject to $x_1 + 2x_2 \le 30$

  and $x_1 \ge 0,\ x_2 \ge 0.$

- In this case

$$c = \begin{bmatrix} 15 & 30 \end{bmatrix}, \quad x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \quad Q = \begin{bmatrix} 4 & -4 \\ -4 & 8 \end{bmatrix}, \quad A = \begin{bmatrix} 1 & 2 \end{bmatrix}, \quad b = [30].$$

Note that

$$x^T Qx = \begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} 4 & -4 \\ -4 & 8 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} (4x_1 - 4x_2) & (-4x_1 + 8x_2) \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

$$= 4x_1^2 - 4x_2x_1 - 4x_1x_2 + 8x_2^2$$

$$= q_{11}x_1^2 + q_{21}x_2x_1 + q_{12}x_1x_2 + q_{22}x_2^2$$

Multiplying through by –(1/2) gives

-(1/2) $\boldsymbol{x^T Qx} = -2x_1^2 + 4x_1x_2 - 4x_2^2$ which is the nonlinear part of the objective function.

Terminology

- Support vector machine: *máy véc tơ hỗ trợ*, hyperplane: *siêu phẳng*, separating hyperplane*: siêu phẳng tách* , binary classification: *phân lớp hai lớp*, multi-class classification: *phân lớp nhiều lớp*, support vector: *véc tơ hỗ trợ*, support hyperplane: *siêu phẳng hỗ trợ*, nonlinear mapping: *ánh xạ phi tuyến*, linearly separable data: *dữ liệu khả tách một cách tuyến tính*, linearly non-separable data: *dữ liệu không khả tách một cách tuyến tính*, margin: *khoảng biên*, maximal marginal hyperplane: *siêu phẳng tách với khoảng biên cực đại*, decision boundary: *biên quyết định*, orthogonal to: *trực giao với*, constrained optimization: *tối ưu hóa có ràng buộc*, Lagrange multiplier: *nhân tử Lagrange*, quadratic programming: *quy hoạch toàn phương*, objective function: *hàm mục tiêu*, constraint: *ràng buộc*, global minimum: *cực tiểu toàn cục*, local minimum: *cực tiểu cục bộ*, separating hypersurface: *siêu diện tách*, kernel function: *hàm nhân*, polynomial kernel: *hàm nhân đa thức*, nonlinear decision boundary: *biên quyết định phi tuyến*, SVM with soft margin: *SVM với khoảng biên mềm*, tradeoff: *sự đánh đổi*, slack variable: *biến bù*, misclassification error: *lỗi phân lớp sai.*